DÉMO 004-E · CHANGEMENTS ENTRE LES ÉTAPES

# Users API

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-users (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-use-fetch-api (2 fichiers)

## Étape 1 etape-1-users : point de départ

fetch dans un useEffect de Users, avec isLoading et isError

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

**Styles fournis (copiés tels quels, pas tapés en live)** : src/App.css (100 lignes)

**Fichiers générés par Vite, non modifiés (non affichés)** : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Users from "./components/Users";

function App() {
  return <Users />;
}

export default App;
```

DÉPART **src/components/Users.jsx** 59 lignes

```jsx
import { useEffect, useState } from "react";

function Users() {
  const [users, setUsers] = useState([]);
  const [isLoading, setIsLoading] = useState(false);
  const [isError, setIsError] = useState(false);

  const getData = async () => {
    setIsLoading(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const json = await response.json();
      setUsers(json);
    } catch (error) {
```

```
      setIsError(true);
    } finally {
      setIsLoading(false);
    }
  };

  // useEffect avec [] : getData tourne une seule fois, au montage
  useEffect(() => {
    getData();
  }, []);

  return (
    <div>
      <h2 className="container">Welcome to users page</h2>
      {isLoading ? (
        <div className="loading">Loading ...</div>
      ) : (
        <table className="table">
          <thead>
            <tr>
              <th className="header">Id</th>
              <th className="header">Name</th>
              <th className="header">Email</th>
            </tr>
          </thead>
          <tbody>
            {users.map((user) => {
              return (
                <tr key={user.id}>
                  <td className="item">{user.id}</td>
                  <td className="item">{user.name}</td>
                  <td className="item">{user.email}</td>
                </tr>
              );
            })}
          </tbody>
        </table>
      )}
    </div>
  );
}

export default Users;
```

## Étape 1 → 2 etape-1-users → etape-2-use-fetch-api

La logique sort dans le custom hook useFetchApi(url) ; Users ne garde que l'affichage

**MODIFIÉ** `src/components/Users.jsx` +7 −24

```
@@ -1,29 +1,12 @@
- import { useEffect, useState } from "react";
+ import useFetchApi from "../hooks/useFetchApi";

  function Users() {
-   const [users, setUsers] = useState([]);
-   const [isLoading, setIsLoading] = useState(false);
-   const [isError, setIsError] = useState(false);
-
-   const getData = async () => {
-     setIsLoading(true);
-     try {
-       const response = await fetch(
-         "https://jsonplaceholder.typicode.com/users"
-       );
-       const json = await response.json();
-       setUsers(json);
-     } catch (error) {
-       setIsError(true);
-     } finally {
-       setIsLoading(false);
-     }
-   };
-
-   // useEffect avec [] : getData tourne une seule fois, au montage
-   useEffect(() => {
-     getData();
-   }, []);
+   // Toute la logique de fetch (data, loading, error) est dans le custom hook useFetchApi
+   const {
+     data: users,
+     isLoading,
+     isError,
+   } = useFetchApi("https://jsonplaceholder.typicode.com/users");

    return (
      <div>
```

**AJOUTÉ** `src/hooks/useFetchApi.js` +31 −0

```
@@ -1,0 +1,31 @@
+ import { useEffect, useState } from "react";
+
+ // Custom hook useFetchApi : la logique de fetch de Users (data, loading, error), réutilisable : seule l'URL change
+ function useFetchApi(url) {
+   const [data, setData] = useState([]);
+   const [isLoading, setIsLoading] = useState(false);
+   const [isError, setIsError] = useState(false);
+
+   const getData = async () => {
+     setIsLoading(true);
+     try {
+       const response = await fetch(url);
+       const json = await response.json();
+       if (!response.ok) {
+         throw new Error("Network response was not ok");
+       }
```

```
+       setData(json);
+     } catch (error) {
+       setIsError(true);
+     } finally {
+       setIsLoading(false);
+     }
+   };
+   useEffect(() => {
+     getData();
+   }, []);
+
+   return { data, isLoading, isError };
+ }
+
+ export default useFetchApi;
```